



UniAtenas

Trabalho de Seminários

Métodos de Ordenação

NF2 – Mapeamento e Desenvolvimento de Banco de Dados

Sistemas de Informação

Prof. Me. Luis Vinicius

2026



Sumário

1	Apresentação	4
2	Organização da turma e grupos	4
3	Algoritmos disponíveis	4
4	Objetivos	5
5	Desenvolvimento do trabalho	6
5.1	Origem e contexto histórico	6
5.2	Funcionamento	6
5.3	Pseudo-código	6
5.4	Teste de mesa	7
6	Implementação	7
7	Análise de complexidade	8
8	Métricas experimentais	8
9	Benchmarks	9
9.1	Observação importante	9
10	Características dos dados de entrada	9
10.1	Vetor ordenado	9
10.2	Vetor em ordem reversa	9
10.3	Vetor aleatório	9
10.4	Vetor parcialmente desordenado	9
10.5	Vetor com elementos repetidos	10
11	Repetição dos experimentos	10
12	Controle experimental	10
13	Gráficos	11
14	Tabelas de resultados	12
15	Comparação entre teoria e experimento	12
16	Validação da implementação	13
17	Comparação opcional com métodos nativos	13
18	Discussão	14
19	Sugestão de estrutura do relatório	14
20	Código e material computacional	15



21 Orientação e auxílio do professor	15
22 Entrega	16
23 Sugestão de estrutura da apresentação	16
24 Critérios de avaliação	17
25 Considerações sobre os benchmarks	17
26 Conclusão	17
27 Referências	18
28 Considerações finais	18



1 Apresentação

Os algoritmos de ordenação constituem uma parte fundamental da Ciência da Computação e das Estruturas de Dados.

Operações de ordenação estão presentes em diversas aplicações computacionais, incluindo bancos de dados, sistemas de busca, processamento de dados, análise de informações e sistemas de tomada de decisão.

Este trabalho tem como objetivo estudar diferentes algoritmos de ordenação sob uma perspectiva **teórica e experimental**.

Cada grupo deverá estudar um algoritmo de ordenação, compreender seu funcionamento, implementar uma versão computacional e avaliar seu comportamento por meio de experimentos.

O trabalho deverá relacionar:

Algoritmo → Funcionamento → Implementação → Complexidade → Experimento → Análise

O objetivo não é apenas implementar o algoritmo, mas compreender **como e por que seu desempenho se modifica conforme as características e o tamanho dos dados de entrada**.

2 Organização da turma e grupos

Grupos entre 3 e 5 pessoas serão formados pelos próprios alunos. Cada grupo deverá escolher **um algoritmo de ordenação** da lista apresentada na Seção 3.

Cada algoritmo deverá ser estudado por apenas um grupo, sempre que o número de grupos permitir.

Todos os integrantes do grupo deverão compreender:

- o princípio de funcionamento do algoritmo;
- o pseudo-código;
- a implementação;
- a análise de complexidade;
- os experimentos realizados;
- os resultados obtidos;
- as conclusões apresentadas.

3 Algoritmos disponíveis

Cada grupo deverá escolher um dos seguintes algoritmos:

- Selection Sort;



- Insertion Sort;
- Cocktail Shaker Sort;
- Gnome Sort.
- Quick Sort;
- Heap Sort;
- Tim Sort;
- Intro Sort.
- Counting Sort;
- Radix Sort;
- Bucket Sort;
- Pigeonhole Sort.
- Shell Sort;
- Comb Sort;
- Cycle Sort;
- Bitonic Sort.

4 Objetivos

O trabalho possui os seguintes objetivos:

- compreender o funcionamento de algoritmos de ordenação;
- estudar diferentes estratégias para ordenar dados;
- desenvolver pseudo-códigos;
- realizar testes de mesa;
- implementar algoritmos em linguagem de programação;
- analisar (mesmo que parcialmente) a complexidade assintótica;
- realizar experimentos computacionais;
- analisar o efeito do tamanho da entrada;
- comparar resultados experimentais com resultados teóricos;
- desenvolver capacidade de análise crítica de algoritmos.



5 Desenvolvimento do trabalho

Para o algoritmo escolhido, o grupo deverá desenvolver as seguintes etapas.

5.1 Origem e contexto histórico

Apresente, quando houver informações disponíveis, a origem do algoritmo e seu contexto histórico.

Explique:

- quando ou em que contexto o algoritmo surgiu;
- seus principais autores ou pesquisadores, quando conhecidos;
- qual problema motivou seu desenvolvimento;
- como o algoritmo se relaciona com outros métodos.

Não é necessário realizar uma pesquisa histórica extensa.

O objetivo é contextualizar o algoritmo.

5.2 Funcionamento

Explique detalhadamente como o algoritmo funciona.

A explicação deverá permitir que um leitor compreenda o processo de ordenação sem precisar consultar outra fonte.

Utilize exemplos pequenos sempre que necessário.

Por exemplo:

[5, 2, 8, 1, 4]

Mostre como o algoritmo transforma a sequência original até chegar ao vetor ordenado.

5.3 Pseudo-código

Apresente o pseudo-código do algoritmo.

O pseudo-código deverá:

- apresentar claramente as entradas;
- apresentar as principais operações;
- utilizar estruturas de repetição e decisão;
- mostrar o processo de ordenação;
- apresentar a saída.

O pseudo-código deverá ser acompanhado de uma explicação das principais etapas.



5.4 Teste de mesa

Realize um teste de mesa utilizando uma entrada pequena.

Recomenda-se utilizar entre 5 e 8 elementos.

Por exemplo:

[7, 3, 5, 1, 6]

O teste deverá mostrar a evolução do vetor durante a execução.

Uma representação possível é:

Etapa	Estado do vetor	Operação
0	[7, 3, 5, 1, 6]	Estado inicial
1		
2		
3		
4		
⋮		
n		Vetor ordenado

O grupo deverá identificar as principais operações realizadas em cada etapa.

Quando possível, apresente também um **diagrama visual** do processo.

6 Implementação

O grupo deverá implementar o algoritmo escolhido.

Recomenda-se a utilização de:

- Linguagem C;
- Python;
- Google Colab;
- Jupyter Notebook.

A implementação deverá ser realizada **pelo próprio grupo**.

Não é suficiente utilizar diretamente:

```
sorted()
```

ou

```
list.sort()
```

para realizar a ordenação.

Essas funções poderão ser utilizadas apenas como referência para validar os resultados.

O código deverá permitir:

1. gerar ou receber um vetor;



2. executar o algoritmo;
3. medir as operações relevantes;
4. medir o tempo de execução;
5. verificar se o resultado está corretamente ordenado.

7 Análise de complexidade

O grupo deverá apresentar a análise assintótica do algoritmo, não necessariamente rigorosa, uma argumentação intuitiva em suas próprias palavras basta.

Deverão ser discutidos, quando aplicável:

- melhor caso;
- caso médio (opcional);
- pior caso;
- complexidade de tempo;
- complexidade de espaço (opcional).

Utilize a notação:

$$O(\cdot)$$

e, quando apropriado, outras notações assintóticas.

O grupo deverá explicar **por que** o algoritmo possui a complexidade apresentada.

Não será suficiente apenas apresentar uma tabela contendo as complexidades.

8 Métricas experimentais

Os benchmarks deverão considerar, quando aplicável:

- tempo de execução;
- número de comparações;
- número de trocas;
- número de movimentações ou cópias;
- número de iterações;
- uso de memória;
- uso de CPU.

Nem todas as métricas serão igualmente adequadas para todos os algoritmos.

O grupo deverá identificar quais métricas são relevantes para o algoritmo estudado.

Sempre que uma métrica não puder ser medida de maneira adequada, o grupo deverá explicar o motivo.



9 Benchmarks

O grupo deverá realizar experimentos para avaliar o comportamento do algoritmo em função do tamanho do vetor de entrada.

Recomenda-se utilizar tamanhos em escala crescente, por exemplo:

$$10^3, \quad 10^4, \quad 10^5, \quad 10^6, \quad 10^7.$$

Entretanto, **não é obrigatório utilizar todos esses tamanhos**.

O limite deverá ser determinado de acordo com a complexidade do algoritmo e com a capacidade computacional disponível.

O tamanho máximo poderá ser reduzido quando o tempo de execução ou o consumo de memória tornar o experimento inviável.

O grupo deverá registrar e justificar os tamanhos utilizados.

9.1 Observação importante

Não é necessário executar todos os algoritmos até o mesmo tamanho de entrada.

Um algoritmo de complexidade quadrática pode se tornar computacionalmente inviável para entradas muito grandes, enquanto algoritmos de complexidade $O(n \log n)$ por exemplo podem suportar entradas significativamente maiores.

Essa diferença constitui parte importante da análise experimental.

10 Características dos dados de entrada

Os experimentos deverão considerar diferentes organizações dos dados.

10.1 Vetor ordenado

Exemplo:

$$[1, 2, 3, 4, 5, \dots, n]$$

10.2 Vetor em ordem reversa

Exemplo:

$$[n, n-1, n-2, \dots, 2, 1]$$

10.3 Vetor aleatório

Os elementos deverão ser distribuídos de maneira pseudoaleatória.

10.4 Vetor parcialmente desordenado

O grupo deverá testar diferentes graus de desordem.

Por exemplo:

- 10% de desordem;



- 25% de desordem;
- 50% de desordem;
- 75% de desordem.

A metodologia utilizada para gerar cada nível de desordem deverá ser explicada.

10.5 Vetor com elementos repetidos

O grupo deverá realizar pelo menos um experimento contendo elementos repetidos.

Por exemplo:

[3, 1, 3, 5, 2, 1, 3, 4, 2].

A quantidade e a distribuição dos valores repetidos deverão ser descritas.

11 Repetição dos experimentos

Sempre que possível, cada experimento deverá ser repetido mais de uma vez.

Recomenda-se realizar pelo menos:

5

execuções para cada configuração.

O grupo deverá utilizar a média dos tempos ou outra medida estatística adequada.

Quando apropriado, também poderão ser apresentados:

- mediana;
- desvio padrão;
- intervalo mínimo–máximo.

O objetivo é reduzir a influência de variações ocasionais do ambiente computacional.

12 Controle experimental

Para permitir resultados mais confiáveis, procure manter as condições experimentais semelhantes.

O grupo deverá registrar:

- ambiente utilizado;
- linguagem de programação;
- características do ambiente do Google Colab ou máquina local, quando utilizada;
- tamanho da entrada;
- tipo de entrada;



- número de repetições;
- parâmetros relevantes do algoritmo.

Quando forem utilizados dados aleatórios, recomenda-se definir uma semente para o gerador de números aleatórios à fim de que o experimento possa ser reproduzido.

Por exemplo:

```
random.seed(42)
```

ou

```
numpy.random.seed(42)
```

Dessa maneira, os experimentos podem ser reproduzidos.

13 Gráficos

Os resultados dos benchmarks deverão ser apresentados juntamente com gráficos.

Recomenda-se apresentar, quando aplicável:

- tempo de execução versus tamanho da entrada;
- comparações versus tamanho da entrada;
- trocas versus tamanho da entrada;
- movimentações versus tamanho da entrada;
- memória versus tamanho da entrada.

Quando houver grandes diferenças entre os valores, considere a utilização de escala logarítmica no eixo y .

Por exemplo:

$$\log_{10}(n).$$

Cada gráfico deverá possuir:

- identificação dos eixos;
- unidades;
- legenda, quando necessária;
- breve interpretação no texto.



14 Tabelas de resultados

O grupo deverá apresentar tabelas contendo os principais resultados dos experimentos.
Um exemplo:

n	Tempo (s)	Comparações	Trocas	Memória
10^3				
10^4				
10^5				
10^6				
10^7				

A tabela deverá ser adaptada às métricas efetivamente coletadas pelo grupo.

15 Comparação entre teoria e experimento

Esta é uma das partes mais importantes do trabalho.

O grupo deverá comparar o comportamento observado nos benchmarks com a complexidade assintótica prevista teoricamente.

Por exemplo, se um algoritmo possui comportamento aproximadamente quadrático em determinado caso, discuta se o gráfico experimental apresenta crescimento compatível.

A análise deverá responder perguntas como:

- O comportamento observado foi compatível com a teoria?
- Em quais situações houve diferença?
- Por que o tempo real não é exatamente igual à função assintótica?
- Como o tamanho da entrada afetou o desempenho?
- Como o grau de desordem afetou o algoritmo?
- Como elementos repetidos afetaram o comportamento?
- Qual foi o impacto do hardware e do ambiente computacional?

É importante compreender que:

$$O(f(n))$$

não representa diretamente o tempo em segundos.

A notação assintótica descreve o comportamento de crescimento da função conforme o tamanho da entrada aumenta.



16 Validação da implementação

Antes de realizar os benchmarks, o grupo deverá verificar se o algoritmo realmente ordena os dados corretamente.

Para isso, deverão ser realizados testes com entradas pequenas.

Por exemplo:

`[5, 2, 8, 1, 4]`

deverá produzir:

`[1, 2, 4, 5, 8]`.

Também deverão ser testados casos como:

- vetor vazio, quando aplicável;
- vetor com um elemento;
- vetor já ordenado;
- vetor em ordem reversa;
- elementos repetidos;
- elementos negativos, quando suportados;
- valores iguais.

Uma função de validação poderá ser utilizada para verificar automaticamente se o resultado está ordenado.

17 Comparação opcional com métodos nativos

O grupo poderá realizar uma comparação adicional com uma função de ordenação disponível na linguagem utilizada.

Em Python, por exemplo:

```
sorted()
```

ou:

```
list.sort()
```

Essa comparação é **opcional**.

Caso seja realizada, deverá ser utilizada apenas como referência experimental.

O grupo deverá explicar que uma implementação industrial ou de biblioteca padrão pode utilizar estratégias muito mais sofisticadas do que uma implementação didática.



18 Discussão

A discussão deverá interpretar os resultados obtidos.

O grupo deverá abordar, quando aplicável:

- vantagens do algoritmo;
- limitações do algoritmo;
- situações em que o algoritmo apresenta bom desempenho;
- situações em que apresenta desempenho ruim;
- influência do tamanho dos dados;
- influência da organização dos dados;
- influência de elementos repetidos;
- custo de memória;
- diferenças entre análise teórica e experimental.

Evite simplesmente descrever os gráficos.

É necessário explicar **por que** os comportamentos observados ocorreram.

19 Sugestão de estrutura do relatório

O relatório deverá ser sucinto e ter como objetivo registrar os principais aspectos do trabalho desenvolvido.

A estrutura abaixo é apresentada como uma **sugestão**:

1. Introdução;
2. Origem e contexto do algoritmo;
3. Funcionamento;
4. Pseudo-código;
5. Teste de mesa;
6. Implementação;
7. Complexidade computacional;
8. Metodologia dos experimentos;
9. Resultados;
10. Gráficos e tabelas;
11. Discussão;



12. Conclusão;

13. Referências.

Os grupos possuem liberdade para adaptar essa estrutura de acordo com as características do algoritmo escolhido.

O relatório não precisa apresentar uma fundamentação teórica extensa. O foco deve estar na compreensão do algoritmo, na implementação e na análise dos resultados experimentais.

20 Código e material computacional

O código desenvolvido para o trabalho deverá ser entregue juntamente com o relatório.

Recomenda-se a utilização de ferramentas como:

- Python;
- Google Colab;
- NumPy;
- Matplotlib;
- outras ferramentas consideradas adequadas pelo grupo.

O grupo possui liberdade para organizar o código da maneira que considerar mais adequada.

Quando utilizado o Google Colab, recomenda-se disponibilizar o link do notebook juntamente com a entrega.

21 Orientação e auxílio do professor

Durante o desenvolvimento do trabalho, os grupos poderão solicitar auxílio do professor.

O professor poderá auxiliar, entre outros aspectos, com:

- implementação;
- Python e Google Colab;
- geração dos dados;
- benchmarks;
- medição de desempenho;
- gráficos;
- análise de complexidade;
- interpretação dos resultados.

Os grupos possuem liberdade para buscar diferentes soluções e ferramentas para o desenvolvimento do trabalho.



22 Entrega

A entrega deverá conter o material produzido pelo grupo, incluindo, conforme aplicável:

- relatório;
- código;
- notebook do Google Colab;
- gráficos e tabelas;
- referências utilizadas.

Os arquivos poderão ser organizados em um arquivo `.zip`, caso necessário.

23 Sugestão de estrutura da apresentação

A apresentação será o principal momento de discussão do trabalho.

A estrutura abaixo é apenas uma **sugestão** para organização dos slides:

1. Introdução;
2. Origem e contexto;
3. Funcionamento do algoritmo;
4. Pseudo-código;
5. Teste de mesa;
6. Implementação;
7. Complexidade;
8. Metodologia dos benchmarks;
9. Resultados;
10. Gráficos;
11. Discussão;
12. Conclusão.

Os grupos poderão modificar, reorganizar ou combinar esses tópicos conforme as características do algoritmo e a abordagem escolhida.

O objetivo principal da apresentação é compartilhar com a turma o algoritmo estudado, seu funcionamento e as conclusões obtidas a partir dos experimentos.



24 Critérios de avaliação

A avaliação considerará:

Critério	Peso
Compreensão e explicação do algoritmo	20%
Pseudo-código e teste de mesa	15%
Implementação computacional	15%
Análise de complexidade	15%
Benchmarks e metodologia experimental	15%
Gráficos, tabelas e resultados	10%
Discussão e conclusão	5%
Organização e referências	5%
Total	100%

25 Considerações sobre os benchmarks

Os benchmarks não deverão ser realizados apenas para produzir números.

O objetivo é utilizar os experimentos para compreender o comportamento do algoritmo.

Por exemplo, considere dois algoritmos hipotéticos:

$$T_1(n) = n^2$$

e

$$T_2(n) = n \log n.$$

Para entradas pequenas, a diferença entre eles pode não ser muito significativa.

Entretanto, conforme n aumenta, a diferença no crescimento pode se tornar muito grande.

Esse comportamento deverá ser observado experimentalmente sempre que possível.

Assim, o grupo deverá relacionar:

Complexidade teórica $\longleftrightarrow$ Comportamento experimental

26 Conclusão

Na conclusão, o grupo deverá apresentar uma síntese dos principais resultados encontrados.

A conclusão deverá responder, entre outras, às seguintes questões:

- O que foi aprendido sobre o algoritmo?
- Quais foram suas principais características?



- Os resultados experimentais foram compatíveis com a análise teórica?
- Em quais tipos de entrada o algoritmo apresentou melhor comportamento?
- Quais foram suas principais limitações?
- O que aconteceria com entradas muito maiores?

O objetivo final é que o estudante seja capaz de compreender que a escolha de um algoritmo de ordenação deve considerar não apenas sua implementação, mas também:

dados + tamanho da entrada + complexidade + memória + características do problema

27 Referências

O grupo deverá utilizar referências confiáveis.

Podem ser utilizados:

- livros de Estruturas de Dados;
- livros de Algoritmos;
- artigos científicos;
- documentação oficial;
- materiais acadêmicos;
- páginas institucionais.

As referências deverão ser citadas ao longo do relatório e apresentadas ao final.

Não serão consideradas referências adequadas apenas páginas sem autoria ou conteúdo sem indicação de fonte.

28 Considerações finais

O principal objetivo desta atividade não é simplesmente produzir uma implementação de um algoritmo de ordenação.

Espera-se que o estudante compreenda a relação entre **algoritmo, dados, complexidade e desempenho**.

O processo esperado é:

Problema → Algoritmo → Implementação → Análise → Experimento → Conclusão

Uma análise adequada deve combinar a compreensão do funcionamento interno do algoritmo com evidências experimentais.

Ao finalizar o trabalho, cada grupo deverá ser capaz de responder:



Como o algoritmo funciona, qual é seu custo computacional, como ele se comporta diante de diferentes entradas e por que os resultados experimentais apresentam esse comportamento?

Bom trabalho!

Em caso de dúvidas, procure o professor. A orientação faz parte do processo de aprendizagem.